

Theme 5. Robotic Manipulation

3D Machine Intelligence (59358-01), Fall 2026 team project topic guide

2026-09-21

3D Machine Intelligence (59358-01), Fall 2026 team project guide. The overall guide and the common rules are in `README.md`. This document covers the four main-body tasks of theme 5 (T08 to T11) and two appendix tasks (A14, A16).

1. Overview

This theme is about **connecting 3D perception to robot action**. The main body has language-guided multi-finger grasp generation (T08), humanoid manipulation where only the wrist terms are adjusted (T09), transfer of one policy across robots (T10), and parallel-jaw grasp generation in clutter (T11); task-oriented grasp selection and human hand retargeting are in the appendix.

Main-body tasks

ID	Task	Actual scope	Default baseline	Status
T08	Language-guided dexterous grasping	Pretrained inference and test-time refinement of language-conditioned multi-finger grasps	DexGYSNet, DextER	Conditional candidate, inference/test-time only
T09	Wrist-guided humanoid manipulation	Adjust only the wrist terms of WristMimic (reset window, reward weight), measure whole-body stability	WristMimic	Conditional candidate
T10	Cross-embodiment robot manipulation	Input and output conversion for a pretrained policy, execution constraints	CrossFormer	Public candidate (real transfer evaluation is conditional), inference/test-time only
T11	Grasp pose generation	Candidate reranking, diversity, and confidence calibration on a trained grasp network	Contact-GraspNet	Conditional candidate

Appendix tasks (section 3)

ID	Task	One-line definition	Robot execution	Research variables	Status
A14	Language-conditioned grasping	Pick the grasp that fits a language instruction and its purpose	None (candidate selection)	Combining language fit with geometric stability, paraphrase robustness, change of purpose	Conditional candidate
A16	Human-to-robot interaction transfer	Convert a human hand keypoint trajectory into a robot hand joint trajectory	None (kinematic optimization)	Per-finger weights during contact, temporal term, joint limits	Public candidate, inference/test-time only

T08 and T10 allow inference/test-time improvement only. T09 trains a policy in Isaac Gym but changes only the wrist-related terms. Training a new action head and large-scale RL are out of scope.

How the neighboring tasks differ

- T08 and T11 both generate grasps. **T08 is conditioned on a language instruction and uses a multi-finger hand; T11 builds parallel-jaw grasp candidates from geometry alone.** A14 picks "the pose that fits this purpose" among candidates that already exist.
- T09 is the only task in this theme that trains a policy. But what changes is **the wrist terms only**, and what is measured is grasp success and **the stability of the humanoid**.
- T10 differs in that the policy already supports several robots. The problem is **correcting the representation differences between robots**.
- A16 is **kinematic optimization** that needs neither a policy nor a simulator.

Common preparation

- **Separate offline metrics from closed-loop success rate.** The collision-free grasp ratio, action error, and fingertip error are not actual manipulation success. Report a success rate only when there is a simulator run.
- The success rate of T08 and the training of T09 need Isaac Gym. **Only teams with a local Linux GPU** should choose them.
- T11 and A14 have old TensorFlow/PyTorch dependencies. Confirm the installation in the first week.

2. Task descriptions (main body)

T08. Language-guided dexterous grasping

TL;DR. Generate a multi-finger grasp that matches a language instruction. Pretrained inference and test-time refinement of DexGYSNet/DexGYSGrasp and DexterER.

Definition. Given an object and a **language instruction** ("hold the drill by the trigger"), generate a **multi-finger grasp** that matches the intent of the instruction and is physically stable.

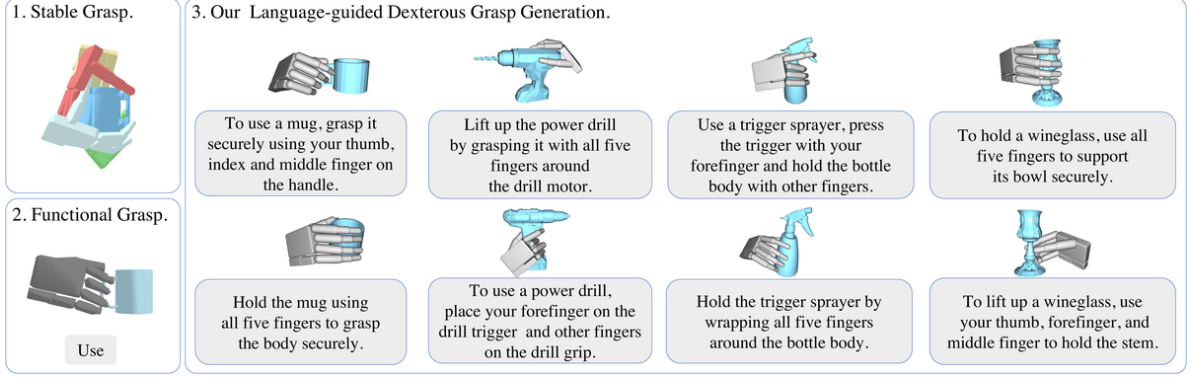


Figure 1. Stable grasps and fixed functional grasps (left) vs. language-guided dexterous grasps (right): the same drill, held by the body or by the trigger, as the instruction says.¹

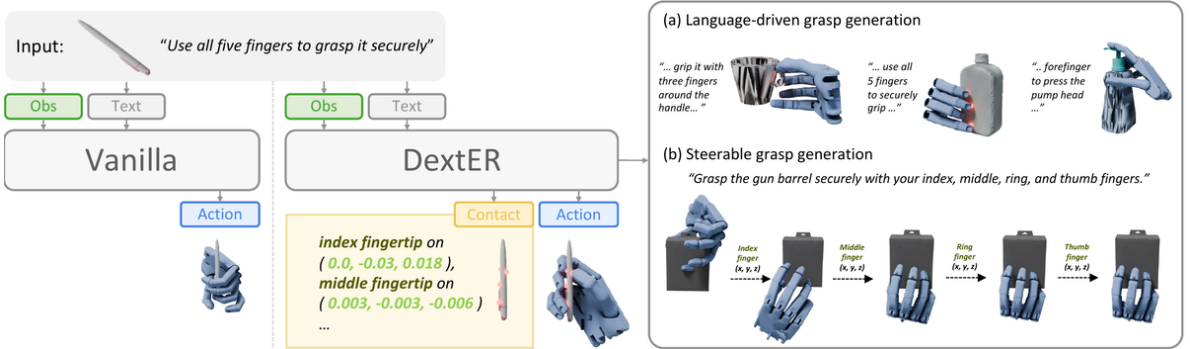


Figure 2. DexterER predicts which finger links touch where (contact tokens) before the grasp, and a partial contact spec steers the grasp, finger by finger.²

Input and output

- Input: an object point cloud or mesh, the instruction, a hand model (ShadowHand in DexGYSNet).
- Output: the hand pose and joint angles. The metrics are success rate, intent alignment, and diversity.
- Key assumption: pretrained inference and test-time refinement only. No retraining on the dataset.
- Example uses: manipulation instructed in language, grasps for tool use (trigger, handle).

Why it is hard

- Intent alignment and grasp quality pull against each other. Diverse grasps that match the instruction penetrate the object; refining for quality moves away from the instruction.
- Typical failures: for "hold the handle" the palm wraps the body. After refinement the fingers go inside the object.

Executable baseline candidates

- **DexGYSNet, DexGYSGrasp** (Wei et al., "Grasp as You Say: Language-guided Dexterous Grasp Generation", NeurIPS 2024, arXiv 2405.19291, code). A dexterous grasp dataset with LLM-assisted language

guidance, and a two-stage generator that aligns intent first and refines quality second.

- Scope: **inference with the public checkpoint** plus test-time grasp refinement (physics-based optimization). No retraining.
- To verify: checkpoint and data access, the hand model files, installation of the simulator for success-rate evaluation (Isaac Gym).
- **DextER** (Lee et al., CVPR 2026, arXiv 2601.16046, code). Embodied reasoning that first predicts, as contact tokens, which finger links touch where on the object surface, then generates the grasp. 67.14% success on DexGYS.
- Conditional candidate. The success rate needs a simulator.

Reference papers

- AffordGrasp (Wu et al., CVPR 2026, arXiv 2603.08021). Grasp synthesis conditioned on affordance. A comparison that conditions on affordance instead of language.

Minimum scope on a small GPU and the first experiment

- Some objects of DexGYSNet (a few dozen) with instructions, one hand model. Inference fits on a single GPU.
- First experiment: with the public checkpoints of both baselines, measure success rate, intent alignment, and diversity on the same objects and instructions. Measure before and after refinement separately.

Example research questions

1. Does a **contact-first intermediate representation** (which link touches where) generalize better than direct grasp regression on **unseen instructions**?
2. At which stage does quality refinement **break the intent**? What does the trade-off curve between refinement strength and intent alignment look like?

Data, metrics, minimum comparisons

- Data: the official DexGYSNet split. Separate unseen objects and unseen instructions.
- Metrics: simulator success rate, intent alignment (part contact agreement), diversity, penetration depth, time.
- Minimum comparisons: DexGYSGrasp as released, DextER as released, the proposed refinement.

Expected deliverables and evaluation mistakes to avoid

- Deliverables: a metric table for both baselines, a curve of refinement strength vs. intent alignment, failure examples (intent mismatch, penetration).
- Caution: **do not use simulator success rate and real-robot success rate as the same thing**. Keep automatic metrics and human ratings of intent alignment apart.

References

- [1] Grasp as You Say: Language-guided Dexterous Grasp Generation, Wei et al., NeurIPS 2024
- [2] DextER: Language-driven Dexterous Grasp Generation with Embodied Reasoning, Lee et al., CVPR 2026
- [3] AffordGrasp: Cross-Modal Diffusion for Affordance-Aware Grasp Synthesis, Wu et al., CVPR 2026

T09. Wrist-guided humanoid manipulation

TL;DR. Transfer a human-object demonstration to a humanoid, adjusting only the wrist terms (reset window, reward weight), and measure grasp success and whole-body stability. WristMimic.

Definition. Transfer a human-object interaction demonstration (body motion, object trajectory, contacts) to a humanoid in a physics simulator. The body and the wrist follow kinematic targets; the fingers get no pose targets and learn to grasp from **object motion and contact outcomes**. In this task **only the wrist-related terms are adjusted** (the reset window, the wrist reward weight, the timing of the contact window). Everything else stays as WristMimic, and you measure how the wrist settings affect grasp success and **the stability of the humanoid**.

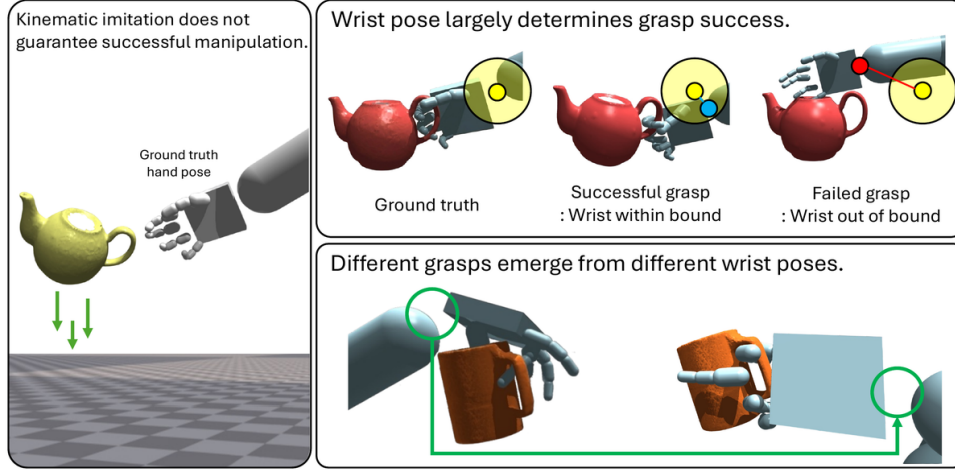


Figure 3. Kinematic imitation is not manipulation: the tracked hand pose drops the teapot (left). The wrist pose decides whether a grasp forms (top right) and which grasp emerges (bottom right).¹

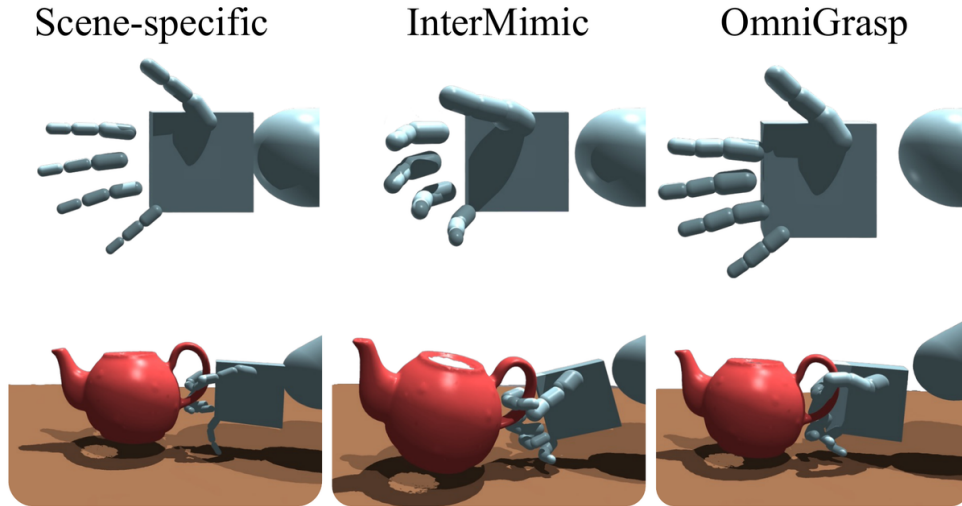


Figure 4. One finger-agnostic formulation, three hand models with different sizes and joint limits, all grasping without finger supervision.¹

Input and output

- Input: a reference sequence (SMPL-X body pose, object trajectory, contact labels), a physics simulator (Isaac Gym), the humanoid and object models.
- Output: a per-scene policy, success rate, object pose error, number of wrist resets, and **stability metrics** (falls, foot slip, joint-limit hits).

- Key assumptions: the wrist is nearly contact-free, so it can be tracked kinematically, and it determines the global placement of the hand. Finger poses are not supervised.
- Example uses: generating humanoid manipulation data from human demonstrations, transfer to robots with a different hand morphology.

Why it is hard

- A position-only hand trajectory carries no contact force. Tracking every finger overconstrains the grasp; removing the wrist constraint means the grasp never forms.
- Typical failures: because of motion-capture error the wrist leaves the graspable region and the object is dropped. When the hand model changes, the finger data no longer fits.

Executable baseline candidates

- **WristMimic** (Yu et al., ECCV 2026, arXiv 2607.06438). The contact-free body and wrist follow kinematic targets; the fingers learn from object tracking and contact rewards. Wrist-specific reset conditions and reward prioritization. PPO in Isaac Gym, 2048 environments, one RTX 3090, one policy per scene. Compared with InterMimic and SkillMimicV2 on ParaHome and OMOMO.
 - Scope: **change only the wrist terms**. One or two scenes, one hand model. The wrist reset thresholds (position, rotation), the wrist reward weight, and the timing of the contact window are the experiment variables.
 - To verify: whether the code is released, Isaac Gym version dependencies, training time per scene. Conditional candidate.

Reference papers

- InterMimic (Xu et al., CVPR 2025, arXiv 2502.20390). Whole-body control for physics-based human-object interaction. The comparison for WristMimic and the origin of its reward structure.
- Related paper: **DexMachina: Functional Retargeting for Bimanual Dexterous Manipulation** (Mandi et al., arXiv 2025, arXiv 2505.24853). Functional retargeting for bimanual dexterous manipulation. A reference for retargeting objectives that preserve contact.

Minimum scope on a small GPU and the first experiment

- One scene (for example, lifting a kettle), PPO with the number of environments reduced to 256 to 512, a short horizon. A local Linux GPU is required.
- First experiment: train one scene with the official setting and measure success rate and object pose error, then change the wrist reset threshold in two steps and measure the same metrics again.

Example research questions

1. How does **the width of the wrist reset window** trade grasp success against **whole-body stability**? Which is more sensitive, the position threshold or the rotation threshold?
2. Does a reward weight that prioritizes the wrist over the upper arm cost **balance** (falls, foot slip)? When the timing of the contact window is moved earlier or later, what breaks first?

Data, metrics, minimum comparisons

- Data: a few sequences of ParaHome or OMOMO. Separate the training scenes from the evaluation seeds.
- Metrics: success rate, object pose error (position, rotation), wrist tracking error, **stability** (fall rate, foot slip, joint-limit hits, torso tilt), training time and VRAM.
- Minimum comparisons: the default WristMimic wrist setting, the setting without wrist constraints, the proposed wrist setting.

Expected deliverables and evaluation mistakes to avoid

- Deliverables: curves of success rate and stability vs. wrist setting, failure examples (grasp failure vs. falling).

- Caution: **simulator success rate is not real-robot success rate.** With per-scene policies and few scenes, do not claim generalization.

References

- [1] WristMimic: Full-Body Humanoid Control with Wrist-Guided Manipulation, Yu et al., ECCV 2026
- [2] DexMachina: Functional Retargeting for Bimanual Dexterous Manipulation, Mandi et al., arXiv 2025

T10. Cross-embodiment robot manipulation

TL;DR. Keep a many-robot policy frozen and correct the observation and action differences between two embodiments. CrossFormer, inference only.

Definition. With a pretrained policy trained on data from several robots kept frozen, correct the differences in observation and action representation between robots (coordinate frames, units, control rates) to improve transfer.

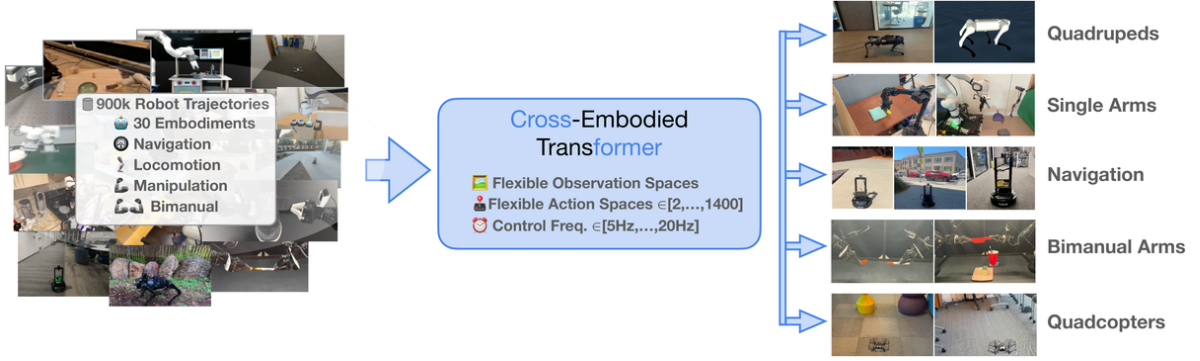


Figure 5. CrossFormer: 900k trajectories from 30 embodiments train one policy that drives arms, quadrupeds, and drones.¹

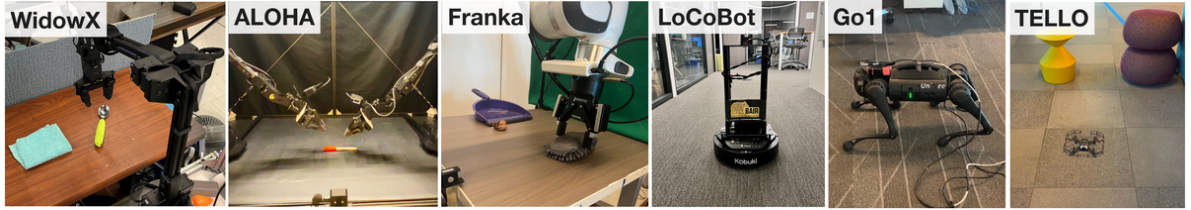


Figure 6. Evaluation embodiments. Two of the arm setups are your starting point.¹

Input and output

- Input: an evaluation subset with observations (images, proprioception) and GT actions for two embodiments the policy supports, plus the robots' kinematics and action conventions.
- Output: corrected actions, per-embodiment action error and execution-constraint violations. Task success if a closed-loop environment exists.
- Key assumptions: the policy already supports that embodiment. Zero-shot transfer to a never-seen robot is not assumed.
- Example uses: deploying one policy on several lab robots, absorbing convention differences between simulated and real robots.

Why it is hard

- Robots differ in end-effector frames, action units (absolute position vs. relative delta), and control rates. The policy output can be an infeasible action (outside joint limits, at a singularity).
- Typical failures: the action scale is off by a factor of two and the robot overshoots the target. Raw MSE over different action units is summed and one robot's error dominates the metric.

Executable baseline candidates

- **CrossFormer** (Doshi et al., CoRL 2024, arXiv 2408.11812, code). A transformer policy trained on 900K trajectories from 20 embodiments. The official **130M pretrained checkpoint** and the `inference_pretrained.ipynb` Colab example are available.

- Scope: **policy frozen**; handle coordinate frames and units, action normalization, and execution constraints. Training a new action head is excluded from this task. Full pretraining is reported as 47 hours on a TPUv5-256 and is reference only.
- Order: confirm offline inference first, then evaluate closed-loop transfer only if a compatible simulator wrapper is available.
- To verify: running policy inference on Colab is not the same as verifying real transfer on two robot environments. Until the wrapper and the data are secured, **state it as an offline feasibility study**.
- Public candidate (Colab inference notebook). Real transfer evaluation is conditional on securing a simulator wrapper. Inference/test-time only.

Reference papers

- AnyBody (arXiv 2505.14986, 2025). A benchmark for designing cross-embodiment evaluation.
- Full pretraining of CrossFormer is reference only.
- Recent paper: **Cross-Embodiment Offline Reinforcement Learning for Heterogeneous Robot Datasets** (Abe et al., ICLR 2026, arXiv 2602.18025). Offline learning from data of different robots. The recent discussion of aligning observation and action spaces.
- Related paper: **UMI-on-Air: Embodiment-Aware Guidance for Embodiment-Agnostic Visuomotor Policies** (Gupta et al., ICRA 2026, arXiv 2510.02614). Keeps an embodiment-agnostic policy frozen and guides it at inference with a controller tracking cost. The same frozen-policy setting as T10.
- Related paper: **One-Policy-Fits-All: Geometry-Aware Action Latents for Cross-Embodiment Manipulation** (Mu et al., ICRA 2026, arXiv 2603.14522). One policy across 11 end-effectors with geometry-aware action latents. A reference for aligning actions across embodiments.

Minimum scope on a small GPU and the first experiment

- A small evaluation subset (some episodes of the corresponding Open X-Embodiment datasets) for two arm-manipulation embodiments the checkpoint supports.
- First experiment: run `inference_pretrained.ipynb` to get the actions of one episode, and record the error against the GT actions normalized per embodiment. Measure latency and VRAM too.

Example research questions

1. Does **end-effector frame alignment and action scaling** (normalization based on per-embodiment statistics) reduce the per-embodiment execution error? On which axes is the difference largest?
2. Can **inverse kinematics and joint-limit projection** reduce infeasible actions? Is a **temporal ensemble** of the same policy output robust to different control rates?

Data, metrics, minimum comparisons

- Data: held-out episodes of the datasets the policy was trained on. Evaluating on training episodes is leakage.
- Metrics: per-embodiment action error (unit-normalized), execution-constraint violations, inference latency. **Add task success rate only when a closed-loop environment exists.** Do not sum raw MSE over different action units.
- Minimum comparisons: the raw output, the proposed correction.

Expected deliverables and evaluation mistakes to avoid

- Deliverables: a per-embodiment error table, a before-and-after comparison of the correction, the distribution of constraint violations.
- Caution: do not present a reduction in offline action error as robot manipulation success. Do not claim zero-shot transfer to an arbitrary never-seen robot.

References

- [1] Scaling Cross-Embodied Learning: One Policy for Manipulation, Navigation, Locomotion and Aviation, Doshi et al., CoRL 2024

- [2] UMI-on-Air: Embodiment-Aware Guidance for Embodiment-Agnostic Visuomotor Policies, Gupta et al., ICRA 2026
- [3] One-Policy-Fits-All: Geometry-Aware Action Latents for Cross-Embodiment Manipulation, Mu et al., ICRA 2026

T11. Grasp pose generation

TL;DR. From a scene point cloud, generate stable 6-DoF parallel-jaw grasp candidates in clutter. Contact-GraspNet.

Definition. Generate **stable 6-DoF parallel-jaw grasp candidates** from the point cloud of an object (or a cluttered scene).

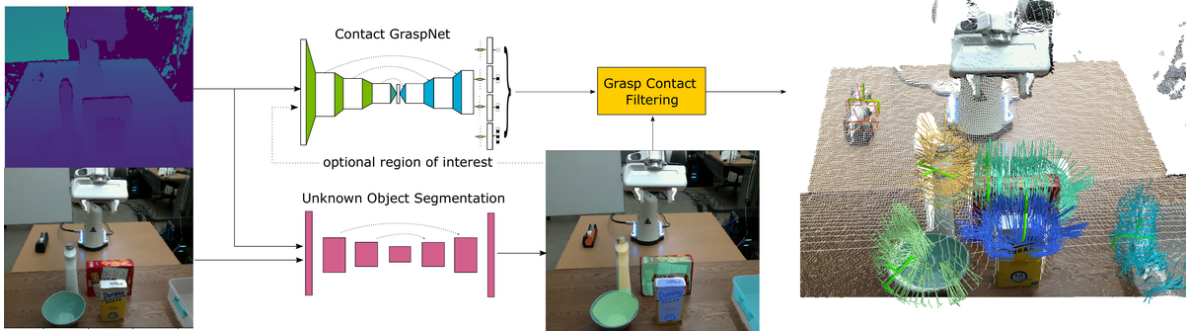


Figure 7. Contact-GraspNet inference: RGB-D scene, unknown-object segmentation, grasp prediction and filtering, grasps drawn on the scene.¹

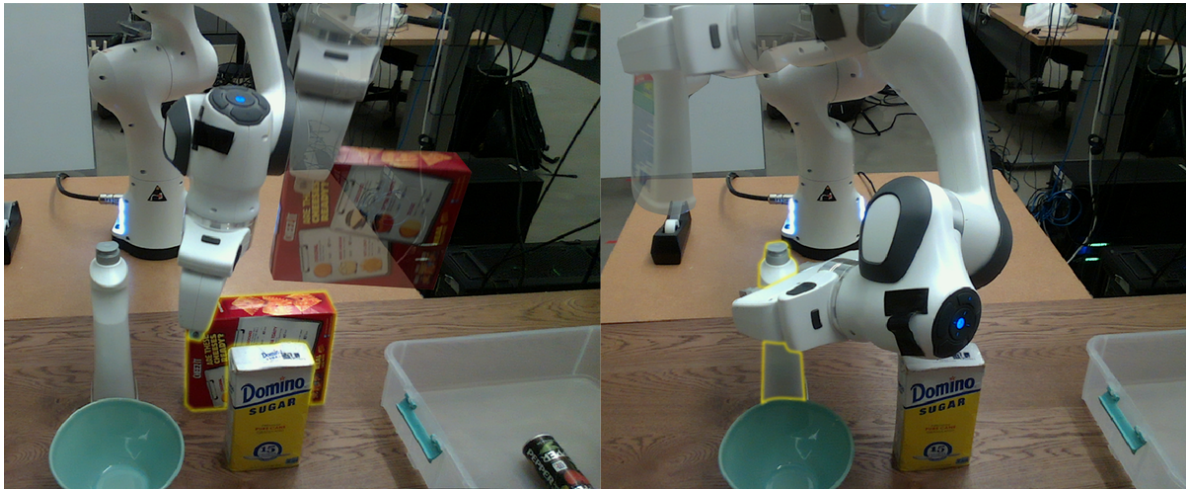


Figure 8. Real grasps in clutter with a parallel-jaw gripper.¹

Input and output

- Input: a scene point cloud (a partial cloud from one depth camera), object segmentation if needed.
- Output: a set of grasp candidates (6-DoF gripper pose, width, score).
- Key assumptions: the gripper is a two-finger parallel gripper of known shape. Objects are rigid.
- Example uses: bin picking, clearing a table, the stage before A14 and robot execution.

Why it is hard

- The cloud is partial, so the back of the object is missing. Two contact points must be estimated from the visible side alone. In clutter, collisions with neighboring objects must be avoided.
- Typical failures: a high-scoring candidate is infeasible because another object blocks the approach path. Candidates cluster on one part of the object, so there is no alternative when that part is occluded. Confidence is overestimated at edges with large depth noise.

Executable baseline candidates

- **Contact-GraspNet** (Sundermeyer et al., ICRA 2021, arXiv 2103.14127, code). Treats each point of the cloud as a contact candidate and regresses grasps directly. Pretrained weights and test data are public.
 - Official guidance: **inference needs a GPU with 8 GB or more; training needs a GPU with 24 GB or more and 64 GB RAM**. In this course pretrained inference is the default and the training path is not recommended as a small-GPU default.
 - Scope: study reranking, filtering, and confidence calibration on top of the candidates of the frozen model.
 - To verify: because of old TensorFlow and the pointnet custom op, **installation on Colab may be difficult**. If installation is blocked, use a geometric grasp sampler (antipodal sampling) as the reduced baseline within the same task.

Reference papers

- The original training of Contact-GraspNet is excluded from the course default scope because of the official 24 GB-or-more guidance.
- Recent paper: **GraspGen-X: Cross-Embodiment 6-DOF Diffusion-based Grasping** (Han et al., CVPR 2026, arXiv 2606.00998). Diffusion-based 6-DoF grasp generation conditioned on a gripper representation. The generative-model line after Contact-GraspNet.
- Related paper: **ActiveGrasp: Information-Guided Active Grasping with Calibrated Energy-based Model** (Lei et al., CVPR 2026, arXiv 2511.12795). Information-guided active grasping with a calibrated energy-based model. Compared directly with Contact-GraspNet in clutter.
- Related paper: **AffordGrasp: Cross-Modal Diffusion for Affordance-Aware Grasp Synthesis** (Wu et al., CVPR 2026, arXiv 2603.08021). Cross-modal diffusion grasp synthesis conditioned on affordance. A recent example of the candidate generation stage.

Minimum scope on a small GPU and the first experiment

- Point-cloud inference one scene at a time, a few objects/scenes (the official test data). Limit the number of candidates and start with a simple geometric filter (collision check between the gripper and the point cloud).
- First experiment: generate candidates on the official test data and record the collision-free ratio, the position and orientation diversity of the candidates, and the inference and verification time.

Example research questions

1. How much does **reranking that accounts for collision and the approach path** raise the ratio of valid (executable) candidates? How does the quality of the top-k change compared with using the score alone?
2. If **position and orientation diversity of candidates is kept** (non-maximum suppression with a diversity criterion), do executable candidates remain in scenes with occlusion? Does calibrating confidence in regions with large depth noise improve stability?

Data, metrics, minimum comparisons

- Data: the official test data, or a small subset of GraspNet-1Billion. Split by scene.
- Metrics: collision-free ratio, grasp coverage/diversity, inference and verification time. If possible, add **success rate under the same simulator conditions**.
- Minimum comparisons: the original Contact-GraspNet score ranking, a simple collision filter, the proposed reranking.

Expected deliverables and evaluation mistakes to avoid

- Deliverables: a curve of the valid ratio among the top-k candidates, a diversity-vs-validity table, visualizations of failed candidates.
- Caution: **collision-free is not the same as actual grasp success**. Do not claim a success rate without a simulator.

References

- [1] Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes, Sundermeyer et al., ICRA 2021

- [2] ActiveGrasp: Information-Guided Active Grasping with Calibrated Energy-based Model, Lei et al., CVPR 2026
- [3] AffordGrasp: Cross-Modal Diffusion for Affordance-Aware Grasp Synthesis, Wu et al., CVPR 2026

3. Appendix: tasks not presented in class

The tasks below were moved out of the main body into the appendix. You may choose them, but the baseline check and the preparation are heavier, or they overlap with a main-body task. The description follows the same structure as the main body.

A14. Language-conditioned grasping

TL;DR. Among existing grasp candidates, pick the one that fits the instruction and the purpose. The public GraspGPT variant.

Definition. Given a **language instruction and a purpose**, such as "hand me the knife", pick the grasp that fits that purpose among the candidates (task-oriented grasping).

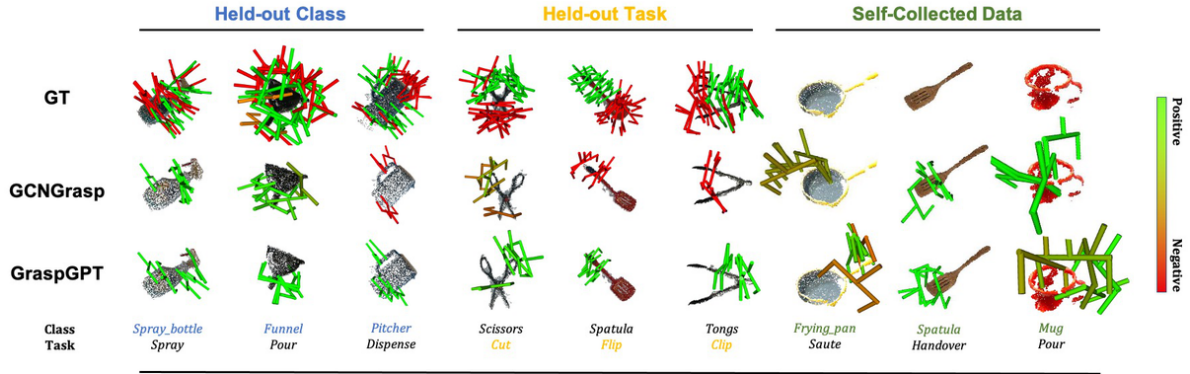


Figure 9. Task-oriented grasps: green fits the task, red does not. Held-out classes, held-out tasks, and self-collected objects.¹

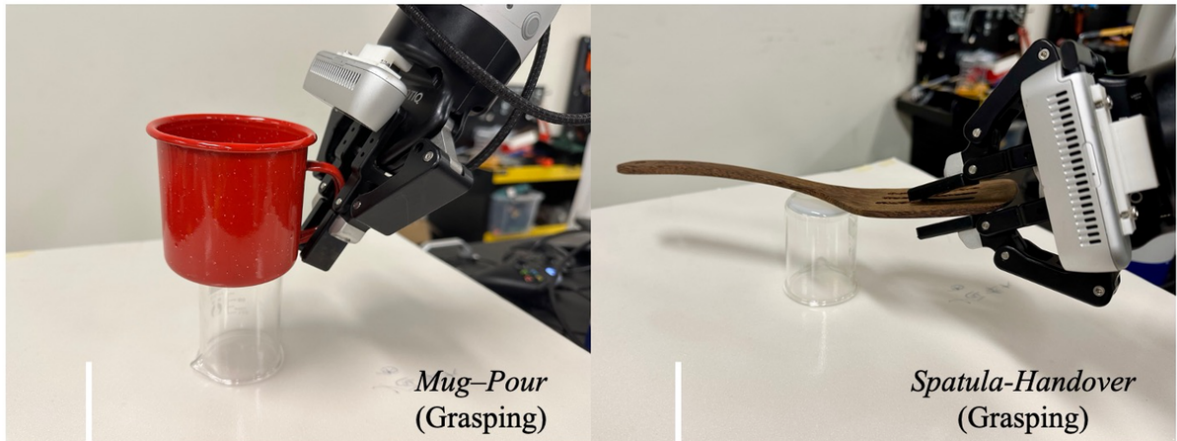


Figure 10. The same object, two instructions: pour from the mug vs. hand over the spatula.¹

Input and output

- Input: an object point cloud, a set of grasp candidates, a language instruction (the task and an object description). LLM-generated object/task descriptions may be prepared in advance.
- Output: a task-fit score per candidate, the final grasp.
- Key assumption: geometrically feasible candidates already exist. The question is which of them fits the purpose.
- Example uses: handing over tools (the handle of a knife toward the person), purpose-specific grasps (holding a cup to drink vs. to wash).

Why it is hard

- Language fit and geometric stability conflict. The part that fits the purpose may be hard to grasp.
- Typical failures: a different phrasing with the same meaning picks a different grasp. When the purpose changes on the same object, the part should change, but the same part is always chosen.

Executable baseline candidates

- **GraspGPT** (Tang et al., IEEE RA-L 2023, arXiv 2307.13204, code). Uses LLM-generated object class and task descriptions as language features, and encodes the grasp candidates together with the point cloud to judge task fit.
 - Mark that the public release is a **lightweight variant of the original**. Using the pre-generated descriptions of the database demo avoids calling an LLM API every time.
 - Scope: start from **frozen scoring/reranking comparisons**, then optionally train a small head only.
 - To verify: checkpoint and old PyTorch dependencies, access to the data (LA-TaskGrasp).

Reference papers

- State the difference between the original GraspGPT and the public lightweight variant as reading scope. Full language data generation and the original training are excluded.
- Recent paper: **AffordGrasp: Cross-Modal Diffusion for Affordance-Aware Grasp Synthesis** (Wu et al., CVPR 2026, arXiv 2603.08021). Grasp synthesis conditioned on affordance. A recent method for choosing grasps that fit language and purpose.
- Recent paper: **DextER: Language-driven Dexterous Grasp Generation with Embodied Reasoning** (Lee et al., CVPR 2026, arXiv 2601.16046, code). Generates multi-finger grasps from a language instruction and a 3D object. The multi-finger version of language-conditioned grasping; also in T08.
- Related paper: **Point2Act: Efficient 3D Distillation of Multimodal LLMs for Zero-Shot Context-Aware Grasping** (Kim et al., ICRA 2026, arXiv 2508.03099). Distills an MLLM into 3D for zero-shot context-aware grasping. Picks the grasp that fits the instruction among AnyGrasp candidates.
- Related paper: **Clutt3R-Seg: Sparse-view 3D Instance Segmentation for Language-grounded Grasping in Cluttered Scenes** (Noh et al., ICRA 2026, arXiv 2602.11660). Sparse-view 3D instance segmentation for language-grounded grasping in clutter. The object separation stage before picking candidates.

Minimum scope on a small GPU and the first experiment

- A small subset of a task-oriented grasp dataset (10 to 20 objects of TaskGrasp or LA-TaskGrasp), existing grasp candidates, cached text features.
- First experiment: measure task-compatible grasp accuracy on the subset with the public checkpoint. As the comparison, pick from the same candidates with the geometric score only.

Example research questions

1. **In what proportion should language fit and geometric stability be combined?** How do task accuracy and geometric feasibility trade off as the combination weight changes?
2. Is the selection stable under **paraphrases** of instructions with the same meaning? On the same object, does it move to the right part **when the purpose changes**?

Data, metrics, minimum comparisons

- Data: the official splits of the TaskGrasp family (both held-out object and held-out task). Check for leakage at the object and task level.
- Metrics: task-compatible grasp accuracy, geometric feasibility, generalization per task/object, time. **Report physical success rate only when there is a simulator run.**
- Minimum comparisons: geometric score only, the public GraspGPT, the proposed combination.

Expected deliverables and evaluation mistakes to avoid

- Deliverables: a sweep curve over the combination weight, a consistency table over paraphrase sets, examples of purpose changes.
- Caution: **do not mix the numbers of the public lightweight release and of the original paper as the same baseline**. If new LLM descriptions are generated, record the cost and the API dependency.

References

- [1] GraspGPT: Leveraging Semantic Knowledge from a Large Language Model for Task-Oriented Grasping,

- Tang et al., IEEE RA-L 2023
- [2] Point2Act: Efficient 3D Distillation of Multimodal LLMs for Zero-Shot Context-Aware Grasping, Kim et al., ICRA 2026
 - [3] DextER: Language-driven Dexterous Grasp Generation with Embodied Reasoning, Lee et al., CVPR 2026
 - [4] Clutt3R-Seg: Sparse-view 3D Instance Segmentation for Language-grounded Grasping in Cluttered Scenes, Noh et al., ICRA 2026

A16. Human-to-robot interaction transfer

TL;DR. Convert a human hand keypoint trajectory into a robot hand joint trajectory. dex-retargeting, CPU optimization, inference only.

Definition. Convert (retarget) the **observed 3D keypoint trajectory** of a human hand into a **joint trajectory** that a robot hand can execute.

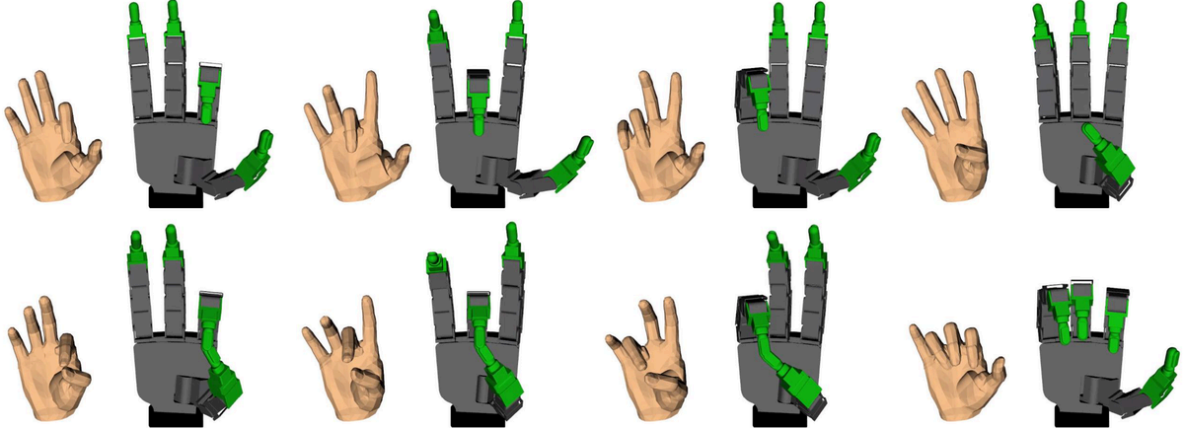


Figure 11. Kinematic retargeting: human hand poses (skin) and the Allegro hand poses that match them (grey and green).¹

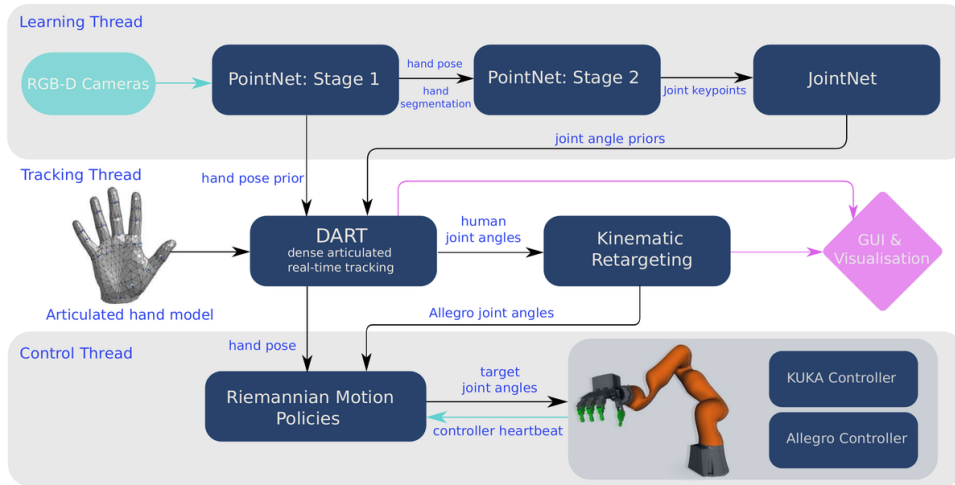


Figure 12. The DexPilot system. Retargeting is one block in the tracking thread; the rest is not part of this task.¹

Input and output

- Input: a time series of human hand keypoints (3D positions of the 21 MANO joints), the kinematic model of the robot hand (URDF). Object pose and mesh if object contact is evaluated.
- Output: a time series of robot hand joint angles, constraint-violation and error metrics.
- Key assumptions: the hand keypoints are given (the camera and the hand pose estimator are not part of this task). The robot hand kinematics are accurate.
- Example uses: teleoperation, creating robot data from demonstration video, moving the output of A13 to a robot.

Why it is hard

- Human and robot hands differ in size, number of joints, and range of motion. Matching fingertip positions pushes joint angles past their limits; matching joint angles misplaces the contacts.

- Typical failures: at the moment of contact the fingertips penetrate or miss the object. Solving each frame separately makes the joints tremble (jitter). Without compensating the hand size difference, small objects cannot be grasped.

Executable baseline candidates

- Method reference: **DexPilot** (Handa et al., ICRA 2020, arXiv 1910.03135). A vision-based teleoperation system. It proposed a retargeting cost that matches the vectors between fingertips.
- Executable implementation: **dex-retargeting** (code). A retargeting library that maps human hand motion to several robot hands. Position-based and vector-based optimization with joint-limit constraints. **Do not equate this implementation with the full original DexPilot system.**
 - Scope: starting from the provided 3D hand keypoints, compare position/vector-based retargeting and the constraints. Inference/test-time only; no trained checkpoint is needed.
 - A reduced task where CPU-centered optimization is possible. The renderer may need a separate environment, so prefer headless numeric evaluation and saved videos.
- Public candidate. It starts with a library install and needs no checkpoint. Inference/test-time only.

Reference papers

- DexMV (ECCV 2022). Background on learning from full demonstrations.
- ContactOpt (CVPR 2021). A reference for designing contact objectives. A related method that may be usable in a lightweight way.
- Recent paper: **DexMachina: Functional Retargeting for Bimanual Dexterous Manipulation** (Mandi et al., ICML 2026, arXiv 2505.24853). Functional retargeting for bimanual dexterous manipulation. A reference for designing retargeting objectives that preserve contact.
- Related paper: **UniDex: A Robot Foundation Suite for Universal Dexterous Hand Control from Egocentric Human Videos** (Zhang et al., CVPR 2026, arXiv 2603.22264). Data and policies retargeted from egocentric human video to 8 dexterous hands. It contains a retargeting pipeline.
- Related paper: **WristMimic: Full-Body Humanoid Control with Wrist-Guided Manipulation** (Yu et al., ECCV 2026, arXiv 2607.06438). When transferring a human demonstration to a humanoid, only the body and the wrist are followed kinematically and the fingers learn from contact outcomes. The baseline of T09.
- Related paper: **ManipTrans: Efficient Dexterous Bimanual Manipulation Transfer via Residual Learning** (Li et al., CVPR 2025, arXiv 2503.21860). Transfers bimanual dexterous manipulation from human to robot with residual learning. A reference for the correction stage after retargeting.

Minimum scope on a small GPU and the first experiment

- 10 to 20 short demonstrations (public hand motion data or self-recorded keypoints), 1 to 2 robot hands (Allegro, Leap, or others the library supports).
- First experiment: run position-based retargeting on one demonstration and record fingertip error, the number of joint-limit violations, time per frame, and joint jerk.

Example research questions

1. If **per-finger weights during the contact phase** are changed (weighting the fingertip error of the contacting fingers more), are contact positions preserved better? How is the contact phase detected?
2. With **temporal regularization and joint limits together**, how much does fingertip error grow while jitter drops? Does normalization for hand size and kinematic differences improve transfer across object sizes?

Data, metrics, minimum comparisons

- Data: public data with hand keypoints (parts of DexYCB, HO-3D) or self-recorded data. For object contact evaluation, use input that also has object pose and mesh. Separate tuning demonstrations from evaluation demonstrations.

- Metrics: fingertip/vector error, joint-limit violations, temporal jerk, contact error and collision if an object exists, time per frame.
- Minimum comparisons: position-based, vector-based, the proposed weighting or regularization.

Expected deliverables and evaluation mistakes to avoid

- Deliverables: an error table per demonstration, a weight sweep, before-and-after videos of the contact phase.
- Caution: **separate simple shape similarity from actual manipulation success**. Do not claim a success rate without an experiment that actually grasps an object in a simulator.

References

- [1] DexPilot: Vision-Based Teleoperation of Dexterous Robotic Hand-Arm System, Handa et al., ICRA 2020
- [2] UniDex: A Robot Foundation Suite for Universal Dexterous Hand Control from Egocentric Human Videos, Zhang et al., CVPR 2026
- [3] WristMimic: Full-Body Humanoid Control with Wrist-Guided Manipulation, Yu et al., ECCV 2026
- [4] ManipTrans: Efficient Dexterous Bimanual Manipulation Transfer via Residual Learning, Li et al., CVPR 2025

4. References

Role	Paper	Links
T11 baseline	Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes (ICRA 2021)	arXiv, code
A14 baseline	GraspGPT: Leveraging Semantic Knowledge from a Large Language Model for Task-Oriented Grasping (IEEE RA-L 2023)	arXiv, code
A16 reference	DexMV (ECCV 2022)	arXiv
A16 method reference	DexPilot: Vision Based Teleoperation of Dexterous Robotic Hand-Arm System (ICRA 2020)	arXiv
A16 implementation	dex-retargeting	code
A16 reference	ContactOpt (CVPR 2021)	arXiv
T10 baseline	CrossFormer: Scaling Cross-Embodied Learning (CoRL 2024)	arXiv, code
T10 reference	AnyBody (2025)	arXiv
A16 recent	DexMachina: Functional Retargeting for Bimanual Dexterous Manipulation (ICML 2026)	arXiv
T10 recent	Cross-Embodiment Offline Reinforcement Learning for Heterogeneous Robot Datasets (ICLR 2026)	arXiv
T11 recent	GraspGen-X: Cross-Embodiment 6-DOF Diffusion-based Grasping (CVPR 2026)	arXiv
A14 recent	AffordGrasp: Cross-Modal Diffusion for Affordance-Aware Grasp Synthesis (CVPR 2026)	arXiv
A14 recent	DexTER: Language-driven Dexterous Grasp Generation with Embodied Reasoning (CVPR 2026)	arXiv, code
T11 related	ActiveGrasp: Information-Guided Active Grasping with Calibrated Energy-based Model (CVPR 2026)	arXiv
A14 related	Point2Act: Efficient 3D Distillation of Multimodal LLMs for Zero-Shot Context-Aware Grasping (ICRA 2026)	arXiv
A14 related	Clutt3R-Seg: Sparse-view 3D Instance Segmentation for Language-grounded Grasping in Cluttered Scenes (ICRA 2026)	arXiv
A16 related	UniDex: A Robot Foundation Suite for Universal Dexterous Hand Control from Egocentric Human Videos (CVPR 2026)	arXiv
A16 related	WristMimic: Full-Body Humanoid Control with Wrist-Guided Manipulation (ECCV 2026)	arXiv
A16 related	ManipTrans: Efficient Dexterous Bimanual Manipulation Transfer via Residual Learning (CVPR 2025)	arXiv
T10 related	UMI-on-Air: Embodiment-Aware Guidance for Embodiment-Agnostic Visuomotor Policies (ICRA 2026)	arXiv

Role	Paper	Links
T10 related	One-Policy-Fits-All: Geometry-Aware Action Latents for Cross-Embodiment Manipulation (ICRA 2026)	arXiv
T09 reference	InterMimic: Towards Universal Whole-Body Control for Physics-Based Human-Object Interactions (CVPR 2025)	arXiv
T08 baseline	Grasp as You Say: Language-guided Dexterous Grasp Generation (NeurIPS 2024)	arXiv, code
T08 baseline	DextER: Language-driven Dexterous Grasp Generation with Embodied Reasoning (CVPR 2026)	arXiv, code
T08 reference	AffordGrasp: Cross-Modal Diffusion for Affordance-Aware Grasp Synthesis (CVPR 2026)	arXiv

Papers without a stated venue are listed with the arXiv year only.